

HDI Predictor: A Comprehensive Measure of Well-Being

Project Description (Abstract):

The Human Development Index (HDI) is a composite statistical measure that captures a country's average achievement across three fundamental dimensions of human development: a long and healthy life, access to knowledge, and a decent standard of living. Traditionally, computing and interpreting HDI requires manually cross-referencing UNDP reports, spreadsheets, and multi-year statistical tables — a process that is slow, error-prone, and inaccessible to non-technical users.

The HDI Predictor addresses this gap with a machine learning web application that harnesses a trained Linear Regression model to instantly estimate a country's HDI score from four measurable indicators: Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI per Capita. The platform includes an HDI Prediction module, which returns a precise numerical score, and a Classification module, which automatically categorizes the result into Low, Medium, High, or Very High HDI — giving users both a precise number and an easy-to-read tier.

Utilizing Scikit-learn's Linear Regression model trained on the Human Development World Index dataset (191 countries) sourced from Kaggle, the HDI Predictor processes user inputs to deliver fast, reliable predictions, achieving an R^2 score of 0.9887 (98.87% accuracy) on the held-out test set. Built with Flask and a dark-themed interactive frontend, the platform ensures a clean, responsive, and user-friendly experience. With a lightweight local deployment and no authentication required, the HDI Predictor empowers policy makers, researchers, and students to explore human development data with confidence and speed.

Problem Statement:

PS-1: A policy maker in a developing country needs a quick, data-driven way to assess a country's HDI level and identify weak areas, but manual analysis of life expectancy, education, and income data is complex and time-consuming, leaving them frustrated and unable to make timely decisions.

PS-2: A researcher studying global development needs to predict HDI categories for countries using simple, measurable indicators, but existing tools require heavy statistical expertise, and raw HDI reports are published as long, non-interactive PDFs.

PS-3: A student learning about global inequality wants to compare human development levels across countries in a simple, hands-on way, but no ML-powered interactive tool exists for exploratory HDI prediction.

Scenarios:

Scenario 1: A policy maker wants a quick snapshot of a country's development level before a budget meeting. They enter the country's Life Expectancy, Schooling years, and GNI per Capita into the form. On clicking Predict, the system instantly returns a result such as “High HDI 0.77,” allowing them to make an informed decision within seconds rather than hours.

Scenario 2: A researcher studying global inequality wants to compare two countries. They run the same four indicators for both countries through the tool and quickly see one lands in “Very High HDI” and the other in “Medium HDI,” giving an immediate comparative insight without reading lengthy UN reports.

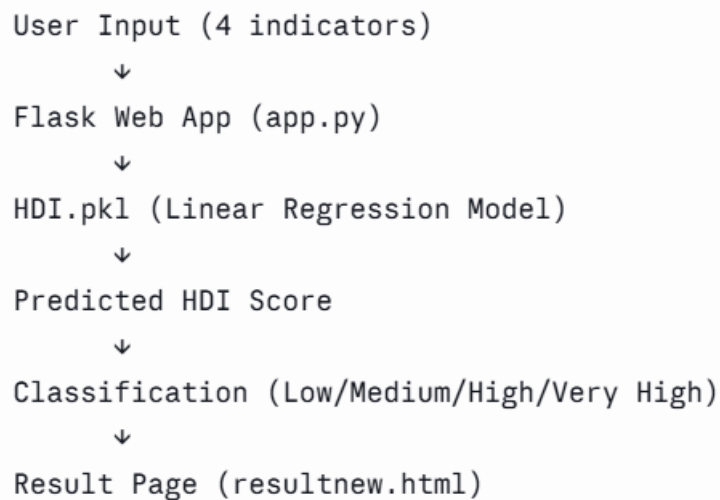
Scenario 3: A student learning about development economics experiments with different input values to see how changing schooling years or GNI shifts the predicted HDI category, gaining hands-on intuition about which factors matter most in determining a country's development tier.

Features:

- HDI Score Prediction — predicts a country's HDI score in real time using a trained Linear Regression model.
- HDI Category Classification — automatically classifies the predicted score into Low (0.30–0.40), Medium (0.40–0.70), High (0.70–0.80), or Very High (0.80–0.94) HDI.
- Interactive Prediction Form — dark-themed, responsive form with example values and range hints for all four indicators.
- Home Page — introduces the HDI concept and the four classification tiers with colour-coded reference cards.
- Result Page — displays the predicted score and category, with options to predict again or return home.

- Data Visualization — the training notebook includes scatter plots and a correlation heatmap explaining the relationship between each indicator and HDI.
- Model Persistence — the trained model is serialized with Pickle (HDI.pkl) so predictions are served instantly without retraining.

Technical Architecture:



Description:

The HDI Predictor uses a modular architecture. The frontend provides a dark-themed, responsive interface built with HTML, CSS, and JavaScript. The Flask backend (“Built With”) handles routing, receives the user's four indicator values (“Receives”), and integrates with the trained HDI.pkl Linear Regression model. The model generates the HDI Score Prediction and the HDI Category Classification, both of which are returned to the user through the result page.

Data Flow: The user submits four indicator values through the prediction form → Flask's /predict route converts the values into a Pandas DataFrame → the DataFrame is passed to the loaded HDI.pkl model → the model returns a predicted HDI score → the score is classified into a category using threshold logic → the final result is rendered on resultnew.html and returned to the user's browser.

The overall data flow of the HDI Predictor begins when the user opens the web application and enters the four required HDI indicator values into the prediction form. After clicking the Predict button, the form data is sent as an HTTP POST request to Flask's /predict route. The backend extracts the submitted values, validates the inputs, converts them into the correct numerical format, and stores them in a Pandas DataFrame that matches the feature structure used during model training.

Tech Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Backend	Python 3.x, Flask
Machine Learning	Scikit-learn (Linear Regression)
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Model Storage	Pickle (HDI.pkl)
Dev Environment	VS Code, Jupyter Notebook
Version Control	Git & GitHub

Pre-requisites:

- **Flask Framework Knowledge:** flask.palletsprojects.com
- **Scikit-learn Familiarity:** scikit-learn.org/stable
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** docs.python.org/3
- **Version Control with Git:** git-scm.com/doc
- **Pandas & NumPy:** pandas.pydata.org/docs, numpy.org/doc/stable
- Jupyter Notebook / VS Code Jupyter Extension

Project Workflow:

Milestone 1: Environment Setup & Dataset Preparation	• Activity 1.1: Install all required Python libraries and dependencies for the project. • Activity 1.2: Set up the project directory structure, including folders for the dataset, Flask app, and training notebook. • Activity 1.3: Download the Human Development World Index dataset from Kaggle.
Milestone 2: Data Exploration & Preprocessing	• Activity 2.1: Load the dataset and inspect its structure, shape, and feature columns. • Activity 2.2: Visualize relationships between HDI and key indicators using scatter plots and a correlation heatmap. • Activity 2.3: Select independent (X) and dependent (y) variables, and handle missing values.
Milestone 3: Model Training	• Activity 3.1: Split the dataset into training (90%) and testing (10%) sets. • Activity 3.2: Train a Linear Regression model on the training data. • Activity 3.3: Evaluate the model using R^2, MAE, MSE and RMSE metrics.
Milestone 4: Model Persistence	• Activity 4.1: Save the trained model as HDI.pkl using Pickle serialization. • Activity 4.2: Verify the saved model by reloading it and running a test prediction.

Milestone 5: Flask Web Application	• Activity 5.1: Write the main application logic in app.py, establishing routes for the home page, prediction form, and predict endpoint. • Activity 5.2: Design and build the frontend templates — home.html, indexnew.html, resultnew.html. • Activity 5.3: Test the complete application locally end-to-end.
Milestone 6: Testing, Conclusion & Future Scope	

Milestone 1: Environment Setup & Dataset Preparation

In this milestone, we focus on preparing the development environment and acquiring the dataset needed to train the HDI prediction model. This involves installing all required Python libraries, organizing the project folder structure, and sourcing a reliable, feature-rich dataset from a trusted public source.

Activity 1.1: Install Required Python Libraries

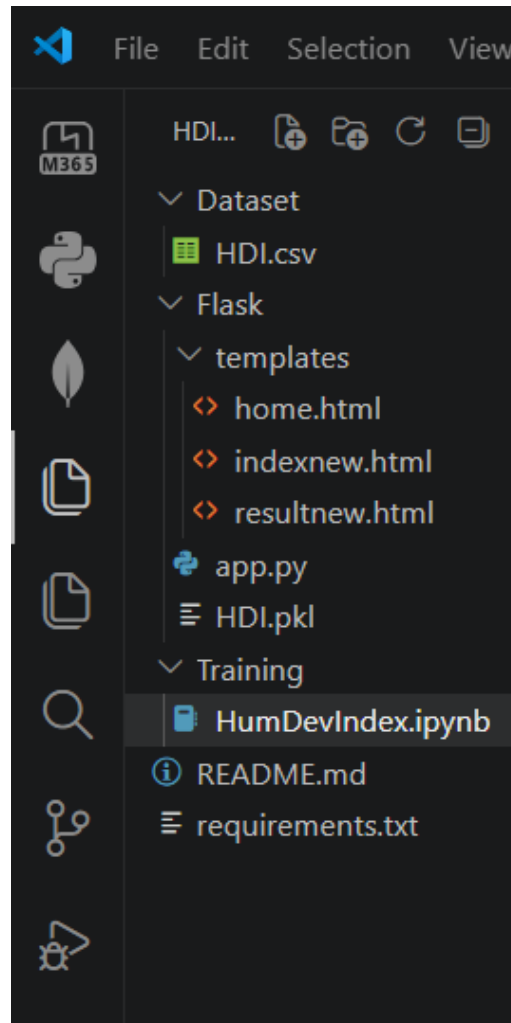
Open VS Code and launch the integrated terminal to install all required Python libraries and dependencies needed for data processing, visualization, machine learning, and the Flask web server.

```

TERMINAL
PS C:\Users\hp\OneDrive\Documents\HDI-Project> pip install numpy pandas matplotlib scikit-learn Flask
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (2.5.0)
Requirement already satisfied: pandas in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (3.0.3)
Requirement already satisfied: matplotlib in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (3.11.0)
Requirement already satisfied: scikit-learn in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (1.9.0)
Requirement already satisfied: Flask in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (3.1.0)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from pandas) (2026.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.10.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (2.22.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: Werkzeug>=3.1 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Flask) (3.1.8)
Requirement already satisfied: Jinja2>=3.1.2 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Flask) (3.1.6)
Requirement already satisfied: itsdangerous>=2.2 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Flask) (2.2.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Flask) (8.3.2)
Requirement already satisfied: blinker>=1.9 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Flask) (1.9.0)
Requirement already satisfied: colorama in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from click>=8.1.3->Flask) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from Jinja2>=3.1.2->Flask) (3.0.3)
Requirement already satisfied: six>=1.5 in C:\Users\hp\AppData\Roaming\Python\Python314\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
PS C:\Users\hp\OneDrive\Documents\HDI-Project>
  
```

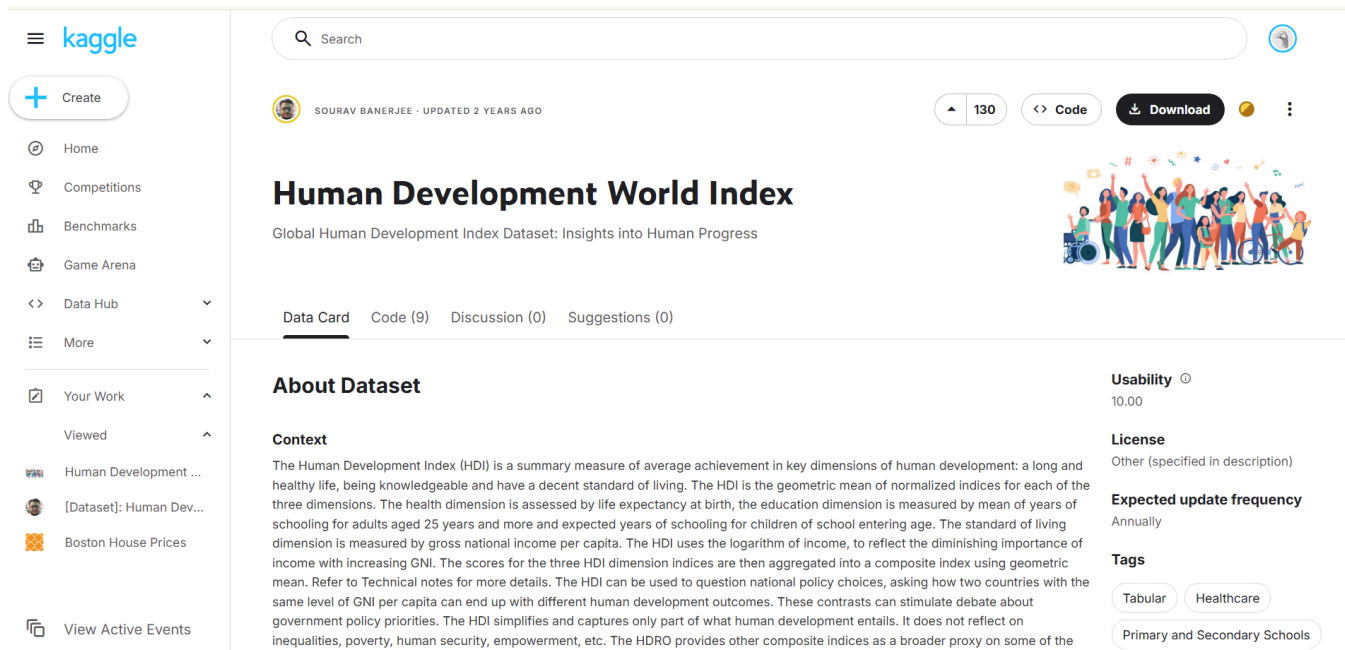
Activity 1.2: Set Up the Project Folder Structure

Create the complete project folder structure, carefully organizing all directories so that datasets, training notebooks, and the Flask application each have a clean, dedicated location. This structure keeps source code, data, and the trained model cleanly separated, which simplifies both development and later deployment.



Activity 1.3: Download and Understand the Dataset

The dataset used for this project is the Human Development World Index dataset, sourced from Kaggle, containing HDI scores and key contributing indicators for 191 countries.



The screenshot shows the Kaggle dataset page for 'Human Development World Index' by Sourav Banerjee, updated 2 years ago. The page has a left sidebar with navigation links: Home, Competitions, Benchmarks, Game Arena, Data Hub, More, Your Work, Viewed, Human Development..., [Dataset]: Human Dev..., Boston House Prices, and View Active Events. The main content area displays the dataset title 'Human Development World Index' with a subtitle 'Global Human Development Index Dataset: Insights into Human Progress'. Below the title are tabs for 'Data Card' (selected), 'Code (9)', 'Discussion (0)', and 'Suggestions (0)'. The 'About Dataset' section includes a 'Context' paragraph explaining the Human Development Index (HDI) as a summary measure of average achievement in key dimensions of human development: a long and healthy life, being knowledgeable and have a decent standard of living. The HDI is the geometric mean of normalized indices for each of the three dimensions. The health dimension is assessed by life expectancy at birth, the education dimension is measured by mean of years of schooling for adults aged 25 years and more and expected years of schooling for children of school entering age. The standard of living dimension is measured by gross national income per capita. The HDI uses the logarithm of income, to reflect the diminishing importance of income with increasing GNI. The scores for the three HDI dimension indices are then aggregated into a composite index using geometric mean. Refer to Technical notes for more details. The HDI can be used to question national policy choices, asking how two countries with the same level of GNI per capita can end up with different human development outcomes. These contrasts can stimulate debate about government policy priorities. The HDI simplifies and captures only part of what human development entails. It does not reflect on inequalities, poverty, human security, empowerment, etc. The HDRO provides other composite indices as a broader proxy on some of the... On the right side, there are metrics: Usability (10.00), License (Other (specified in description)), Expected update frequency (Annually), and Tags (Tabular, Healthcare, Primary and Secondary Schools).

Milestone 2: Data Exploration & Preprocessing

This milestone covers loading the dataset into a Pandas DataFrame, visualizing relationships between HDI and its key contributing factors, and preparing clean input features for model training. Exploratory analysis at this stage confirms that the chosen indicators are meaningful predictors before any model is trained.

Activity 2.1: Load and Inspect the Dataset

The dataset contains 191 rows (one per country) and 6 columns: Country, HDI, Life expectancy, Expected years of schooling, Mean years of schooling, and Gross national income (GNI) per capita. Each row represents a single country's most recent recorded human development indicators.

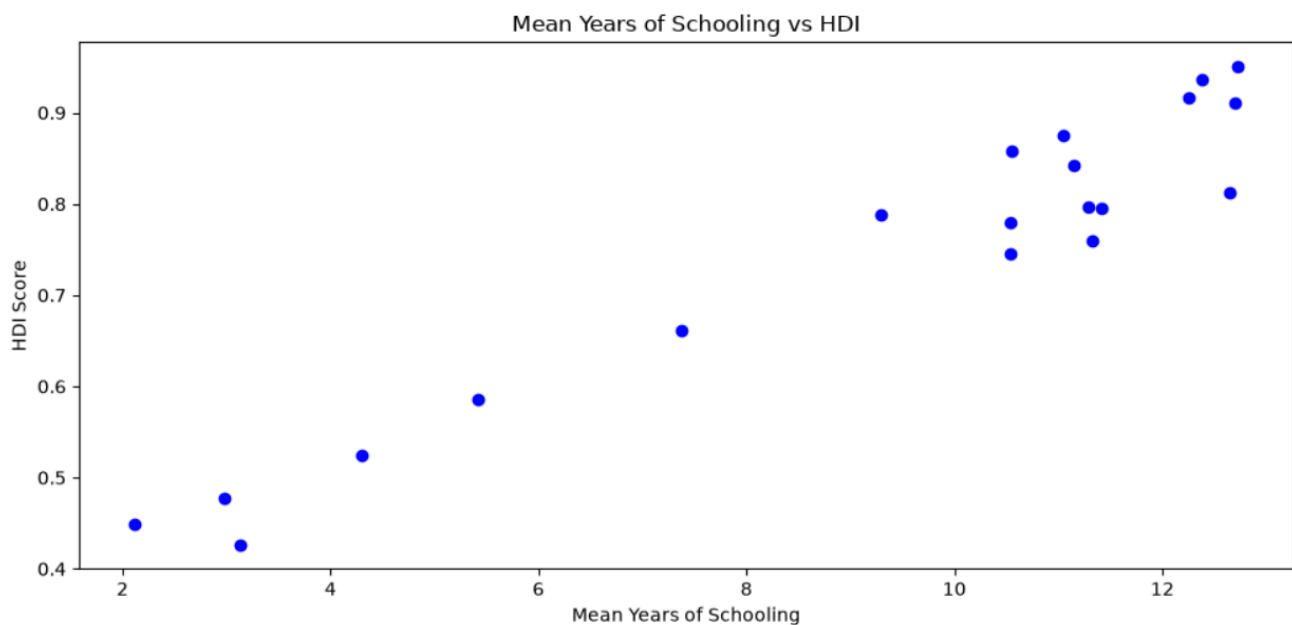
✓ STEP 1: Dataset loaded!

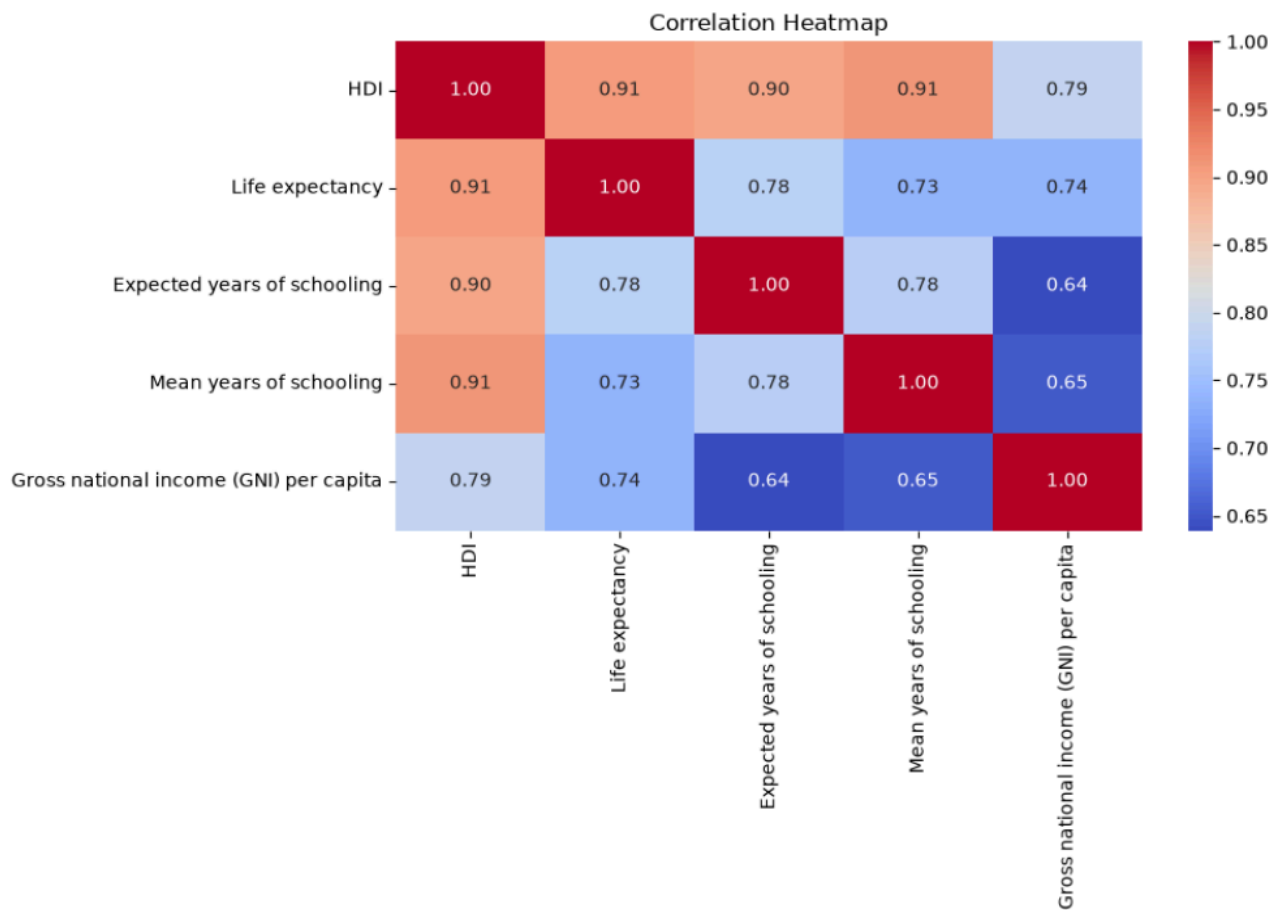
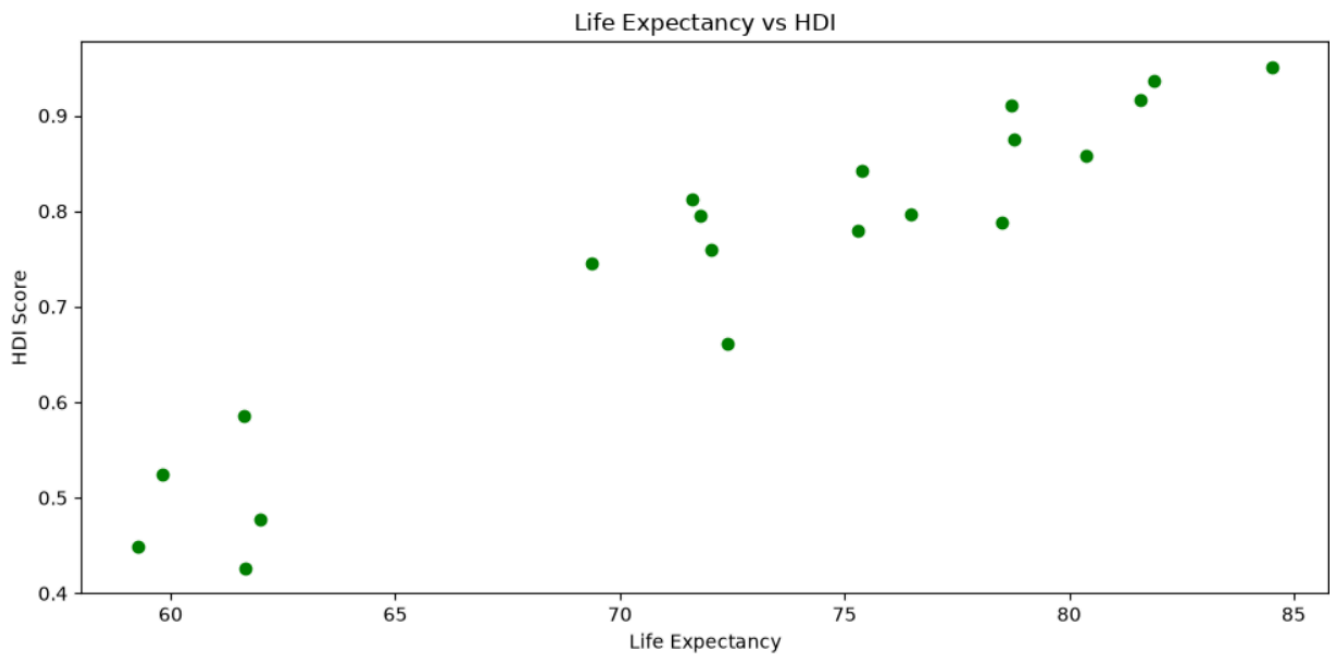
Shape: (191, 6)

	Country	HDI	Life expectancy	Expected years of schooling \
0	Afghanistan	0.478	61.9824	10.263844
1	Angola	0.586	61.6434	12.172100
2	Albania	0.796	76.4626	14.448000
3	Andorra	0.858	80.3684	13.300239
4	United Arab Emirates	0.911	78.7104	15.717690

Activity 2.2: Data Visualization

Two scatter plots and a correlation heatmap were generated to understand how strongly each indicator relates to the HDI score, and to justify the choice of the four features used for training.





Activity 2.3: Feature Selection, Null Handling & Train/Test Split

Independent variables (Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, GNI per Capita) and the dependent variable (HDI) were selected. Missing values in the feature columns were imputed using the column mean, ensuring no rows needed to be dropped. These preprocessing steps, along with the train/test split, were executed together in a single consolidated notebook cell:

```
✓ Plot 3 done!

✓ STEP 3: Features selected!
X shape: (191, 4)
✓ STEP 4: Null values handled!
Life expectancy                0
Expected years of schooling    0
Mean years of schooling        0
Gross national income (GNI) per capita  0
dtype: int64

✓ STEP 5: Data split!
Train: (171, 4)
Test : (20, 4)
```

The output confirms 4 selected features (X shape: 191×4), zero remaining null values after imputation, and a 90/10 train-test split, resulting in 171 training rows and 20 testing rows.

Milestone 3: Model Training

With clean, prepared features, this milestone fits a Linear Regression model on the training data and evaluates its predictive performance on the held-out test set to confirm the model generalizes well to unseen countries.

Activity 3.1 & 3.2: Train the Linear Regression Model

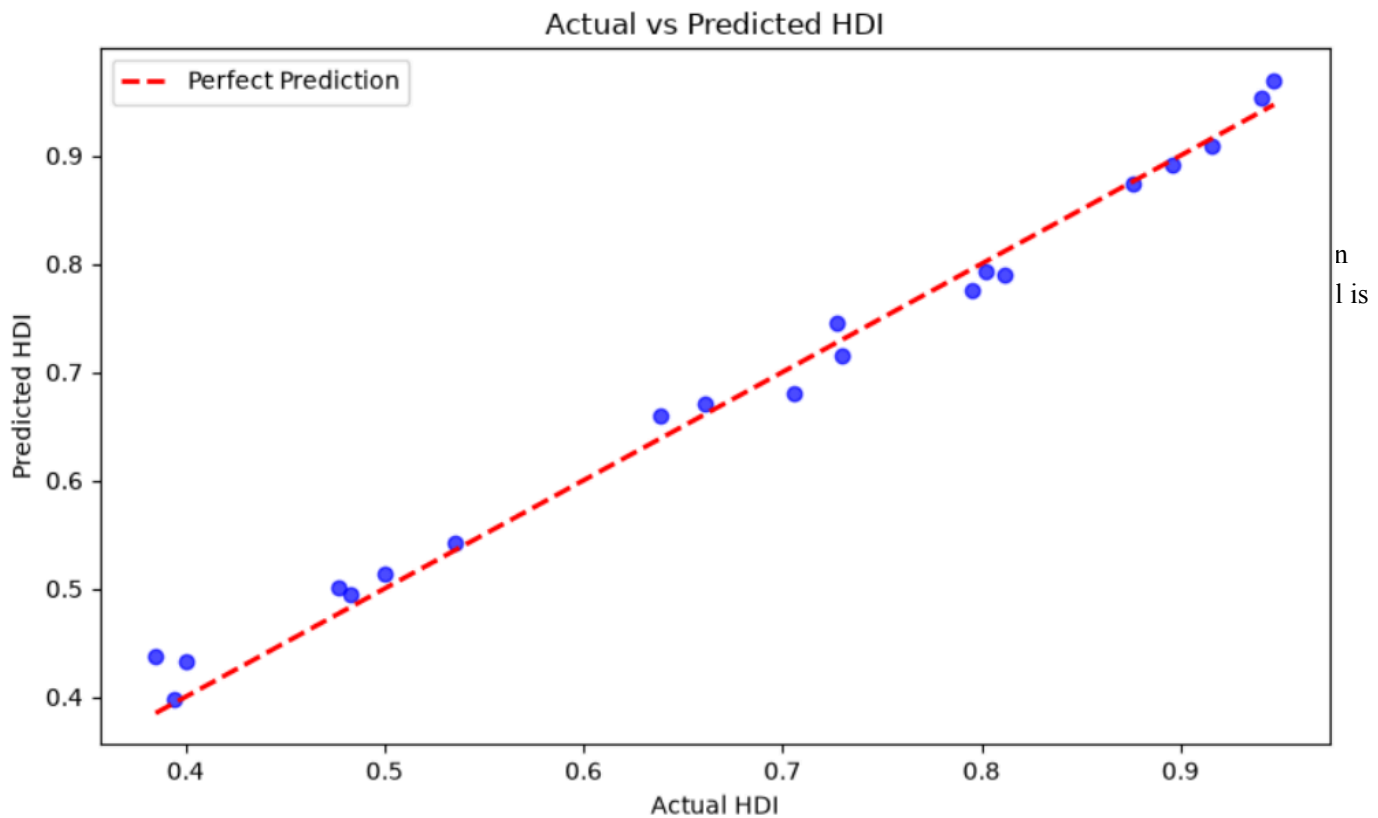
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42)

reg = LinearRegression()
reg.fit(x_train, y_train)
```

Activity 3.3: Predict and Evaluate

The model achieves an R^2 score of 0.9887, a Mean Absolute Error of 0.0166, a Mean Squared Error of 0.0004, and an RMSE of 0.0201 on the test set — an overall accuracy of 98.87%. This indicates the four selected indicators explain almost all of the variance in HDI scores, making Linear Regression a strong and interpretable choice for this problem.



Milestone 4: Model Persistence

After validating the model's accuracy, the trained model is serialized so it can be reused by the Flask application without retraining on every request. This step is what makes real-time prediction on the web possible — the model is trained once, saved to disk, and simply loaded into memory whenever the Flask app starts.

Activity 4.1: Save the Trained Model

```
# — STEP 10: SAVE MODEL —————  
pickle.dump(reg, open('../Flask/HDI.pkl', 'wb'))  
print('\n✅ STEP 10: Model saved as HDI.pkl!')
```

Activity 4.2: Verify the Saved Model

```
# Verify  
loaded = pickle.load(open('../Flask/HDI.pkl', 'rb'))  
test    = loaded.predict([[72.0, 10.5, 5.2, 3341.0]])  
print('✅ Verified! Test prediction:', round(test[0], 4))  
print('\n🎉 ALL DONE! PROJECT COMPLETE!')
```

Milestone 5: Flask Web Application

This milestone turns the trained model into an accessible web application, with a home page, an interactive prediction form, and a result page. The Flask backend is intentionally kept simple — three routes handle the entire user journey from landing page to prediction result.

Activity 5.1: app.py — Imports and Model Loading

```
lask > 📄 app.py > ...  
1  # importing the necessary dependencies  
2  import numpy as np  
3  import pandas as pd  
4  from flask import Flask, render_template, request  
5  import pickle  
6
```

app.py — Core Routes

```
@app.route('/')
def home():
    return render_template('home.html')

@app.route('/Prediction', methods=['POST', 'GET'])
def prediction():
    return render_template('indexnew.html')

@app.route('/Home', methods=['POST', 'GET'])
def my_home():
    return render_template('home.html')

@app.route('/predict', methods=['POST'])
def predict():
```

app.py — Reading Form Inputs

```
# Reading inputs from form
input_features = [float(x) for x in request.form.values()]
features_value = [np.array(input_features)]
```

app.py — Prediction and HDI Classification Logic

```
df = pd.DataFrame(features_value, columns=features_name)

output = model.predict(df)
y_pred = round(float(output[0]), 2)

print('Predicted HDI:', y_pred)

if y_pred >= 0.3 and y_pred < 0.4:
    result = 'Low HDI ' + str(y_pred)
elif y_pred >= 0.4 and y_pred < 0.7:
    result = 'Medium HDI ' + str(y_pred)
elif y_pred >= 0.7 and y_pred < 0.8:
    result = 'High HDI ' + str(y_pred)
elif y_pred >= 0.8 and y_pred <= 0.94:
    result = 'Very High HDI ' + str(y_pred)
else:
    result = 'Out of range - check your input values'

return render_template('resultnew.html', prediction_text=result)
```

Activity 5.2: Build the Frontend Templates

- home.html — introduces the HDI concept and the four classification tiers.
- indexnew.html — the interactive prediction form with four input fields and range hints.
- resultnew.html — displays the predicted score and category, with Predict Again / Home buttons.

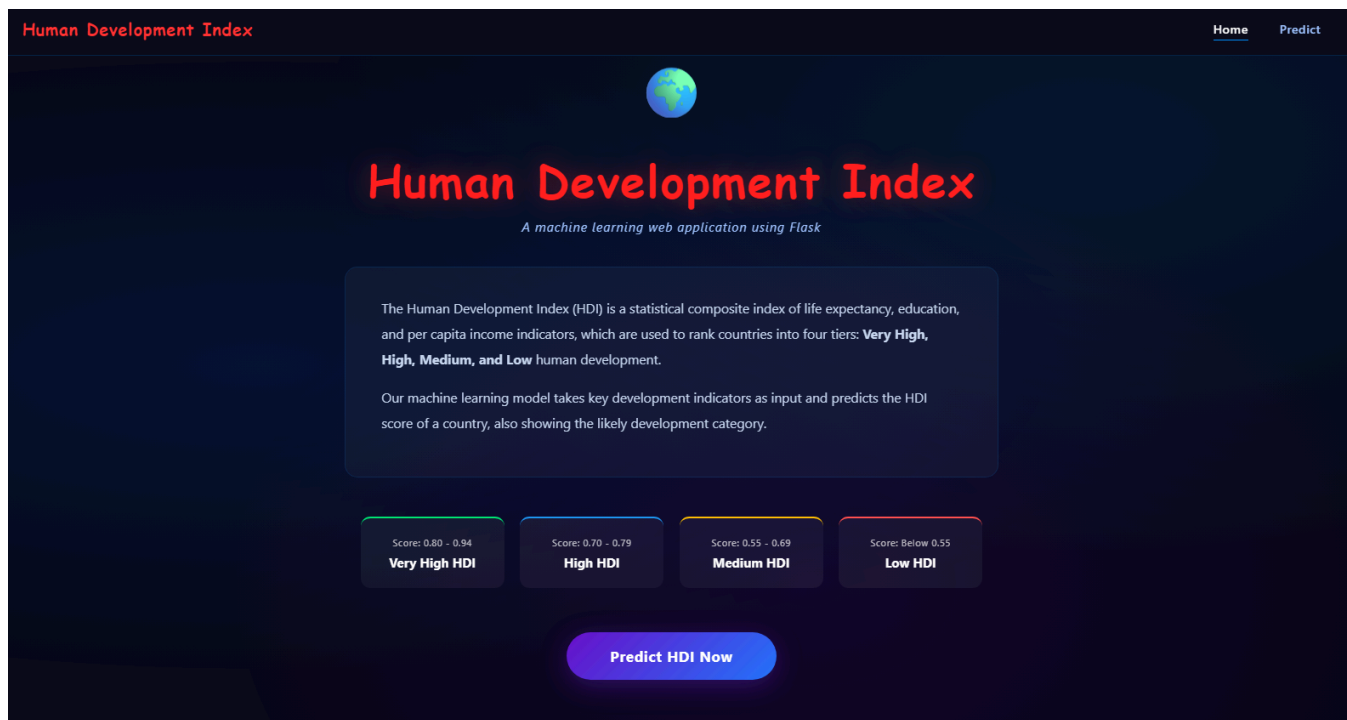
Activity 5.3: Local Testing and Verification

Run the application locally and confirm the Flask development server starts successfully before testing the full user journey through the browser.

```
PS C:\Users\hp\OneDrive\Documents\HDI-Project> cd Flask
PS C:\Users\hp\OneDrive\Documents\HDI-Project\Flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 540-996-146
```

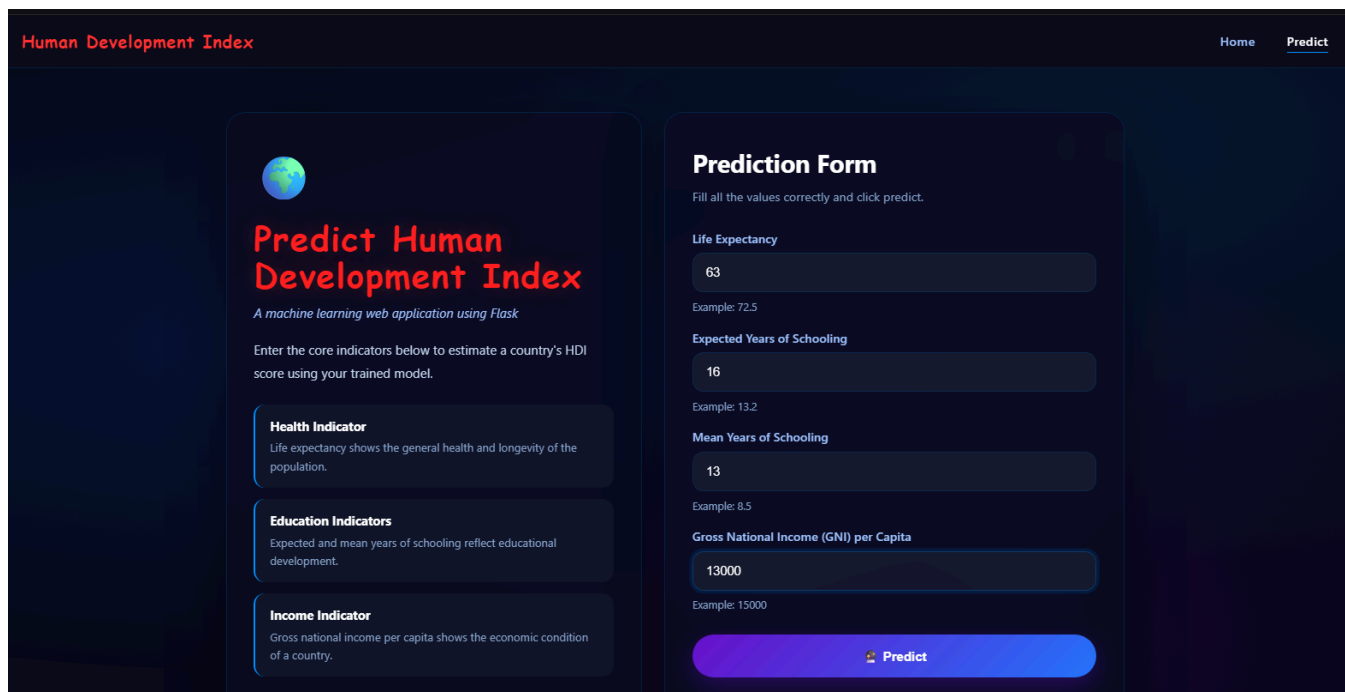
Exploring the Website's Web Pages:

Home Page



Description: The home page introduces the Human Development Index concept and explains how life expectancy, education, and per-capita income combine into a single composite score. It displays the four HDI classification tiers — Very High, High, Medium and Low — as colour-coded reference cards, and includes a “Predict HDI Now” button that takes the user to the prediction form.

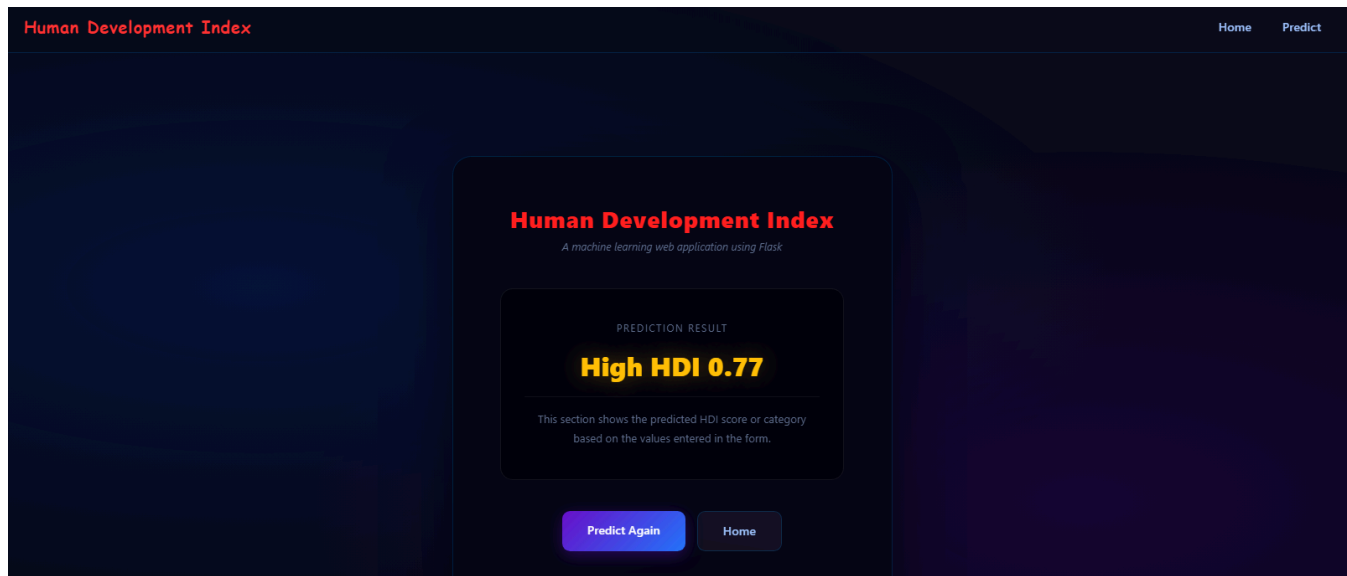
Prediction Form Page



The screenshot shows a web application titled "Human Development Index" with a navigation bar containing "Home" and "Predict" links. The main content area is divided into two columns. The left column features a globe icon, the title "Predict Human Development Index", and a subtitle "A machine learning web application using Flask". Below this, it instructs the user to "Enter the core indicators below to estimate a country's HDI score using your trained model." Three indicators are listed: "Health Indicator" (Life expectancy shows the general health and longevity of the population), "Education Indicators" (Expected and mean years of schooling reflect educational development), and "Income Indicator" (Gross national income per capita shows the economic condition of a country). The right column is titled "Prediction Form" and includes the instruction "Fill all the values correctly and click predict." It contains four input fields: "Life Expectancy" (value: 63, example: 72.5), "Expected Years of Schooling" (value: 16, example: 13.2), "Mean Years of Schooling" (value: 13, example: 8.5), and "Gross National Income (GNI) per Capita" (value: 13000, example: 15000). A large blue "Predict" button is at the bottom right.

Description: The prediction page presents a two-column layout. The left column explains the three underlying dimensions of HDI — Health, Education and Income indicators. The right column contains the input form itself, where the user enters Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI per Capita, each with an example value shown as a hint, before clicking Predict.

Result Page



Description: After submission, the result page displays the predicted HDI score alongside its classified category — in this example, “High HDI 0.77.” The user can click “Predict Again” to test another set of values or “Home” to return to the introduction page.

Milestone 6: Testing & Results

To validate that the application behaves correctly across the full HDI range, several representative test cases were run through the prediction form, covering each of the four classification tiers.

Sample Input (Life Exp /Expected Schooling Yrs /Mean Yrs schooling/ GNI)	Predicted Output
69 / 15 / 12 / 12,000	High HDI 0.77
59.8 / 8.6 / 3.0 / 1,200	Low HDI
72.4 / 11.5 / 5.4 / 9,500	Medium HDI
84.5 / 17.5 / 13.0 / 66,000	Very High HDI

Known Issues / Limitations

- The Flask development server is single-threaded and not production-optimized; this is acceptable for local demo and evaluation but not for public deployment at scale.
- The model is trained on a single reference year of data; predictions do not account for year-over-year trends.
- Out-of-range input values (e.g. extremely high GNI) are still processed and may fall outside the defined HDI category thresholds; the app currently returns a fallback message in this case.

Future Scope

- Support multi-year HDI data instead of a single reference year, allowing trend-based predictions.
- Add more granular indicators, such as the Inequality-adjusted HDI or Gender Development Index.
- Deploy to a cloud platform (Render, Heroku, or AWS) for public access beyond localhost.
- Add user authentication to allow saving and comparing prediction history over time.
- Introduce a country dropdown to auto-fill known indicator values for quick lookup and comparison.

Team Member Details

Name	Role	Contribution
J.Rohit Venkata Manikanta	Team Lead	Model training, Flask backend, frontend, All contribution.
M.Jagan Surya Satish Reddy	Member	Data Set Collection

Conclusion:

A Comprehensive Measure of Well-Being provides a holistic view of quality of life by evaluating multiple dimensions that influence an individual's overall welfare, rather than relying solely on traditional economic indicators such as income or GDP. It encompasses factors including health, education, income, and overall life satisfaction.

By integrating these diverse aspects, this ML-powered HDI Predictor offers policy makers, researchers, and students a fast, accessible way to estimate and classify human development levels, replacing hours

of manual analysis with an instant, interactive prediction. The project journey — from dataset sourcing and exploratory analysis through model training (achieving 98.87% accuracy), persistence, and Flask deployment — demonstrates a complete, reproducible machine learning workflow.

Looking ahead, the platform has clear scope for expansion, from richer multi-year datasets to full cloud deployment, positioning it to grow from a classroom project into a genuinely useful public tool for exploring human development around the world.

Repository:

https://github.com/Rohit12223/AIML-TRACK-_INTERNSHIP

Demo Link:

<https://drive.google.com/file/d/1SvnUi36nXKS2w7iKKkh6T4QuWu0EOFYG/view?usp=sharing>